

Challenge Lab: Amazon S3

NAME: KEVIN CARL B. RICAFORT

COHORT: PHMAN29

Lab Overview

In this challenge lab, you create an Amazon Simple Storage Service (Amazon S3) bucket and perform some routine tasks, such as uploading objects and configuring permissions to make those objects publicly accessible through a browser.

Objectives

By the end of this challenge, you should be able to do the following:

- Create an S3 bucket.
- Upload an object into this bucket
- Access the object by using a web browser.
- List the contents of the S3 bucket by using the AWS Command Line Interface (AWS CLI).

Lab Proper

In this challenge lab using the Amazon S3, I will try to simulate the AWS Management Console using `floci` and `Terraform` in order to practice IaC and some scripting.

Table of Specifications

Key	Value
source	hashicorp/aws
version	5.0
region	us-west-2
access_key	test
secret_key	test

Part 1: Create the Terraform Configuration

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# 1. Point the AWS provider to local Floci
provider "aws" {
  region          = "us-east-1"
  access_key      = "test"
  secret_key      = "test"
  skip_credentials_validation = true
  skip_metadata_api_check    = true
}
```

```

skip_requesting_account_id = true
s3_use_path_style          = true

endpoints {
  s3 = "http://localhost:4566"
}
}

# 2. Task requirement: Create an S3 bucket
resource "aws_s3_bucket" "challenge_bucket" {
  bucket = "my-challenge-bucket"
}

# Allow ACLs on the bucket (needed to make individual objects public)
resource "aws_s3_bucket_ownership_controls" "ownership" {
  bucket = aws_s3_bucket.challenge_bucket.id
  rule {
    object_ownership = "BucketOwnerPreferred"
  }
}

# Allow public ACLs on this bucket
resource "aws_s3_bucket_public_access_block" "public_access" {
  bucket = aws_s3_bucket.challenge_bucket.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls     = false
  restrict_public_buckets = false
}

# 3. Task requirement: Upload an object into the bucket
# (Set acl = "public-read" to make the object publicly accessible)
resource "aws_s3_object" "sample_file" {
  depends_on = [
    aws_s3_bucket_ownership_controls.ownership,
    aws_s3_bucket_public_access_block.public_access
  ]

  bucket      = aws_s3_bucket.challenge_bucket.id
  key         = "hello.html"
  content     = "<h1>Hello from Public S3 in Floci!</h1>"
  content_type = "text/html"
  acl        = "public-read" # Task requirement: Make the object publicly accessible
}

# Output the browser URL
output "object_url" {
  value = "http://localhost:4566/${aws_s3_bucket.challenge_bucket.id}/${aws_s3_object.sample_file.key}"
}

```

Part 2: Deploy with Terraform

In order to deploy the Terraform configuration, use the terminal and enter the following command:

```

terraform init
terraform apply -auto-approve

```

```
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\User\Desktop\Curveball Project\30 DAYS OF PYTHON> terraform apply -auto-approve
```

Figure 1. Terminal output showing the successful initialization of Terraform with the AWS provider, immediately followed by the execution of terraform apply -auto-approve to provision the defined infrastructure.

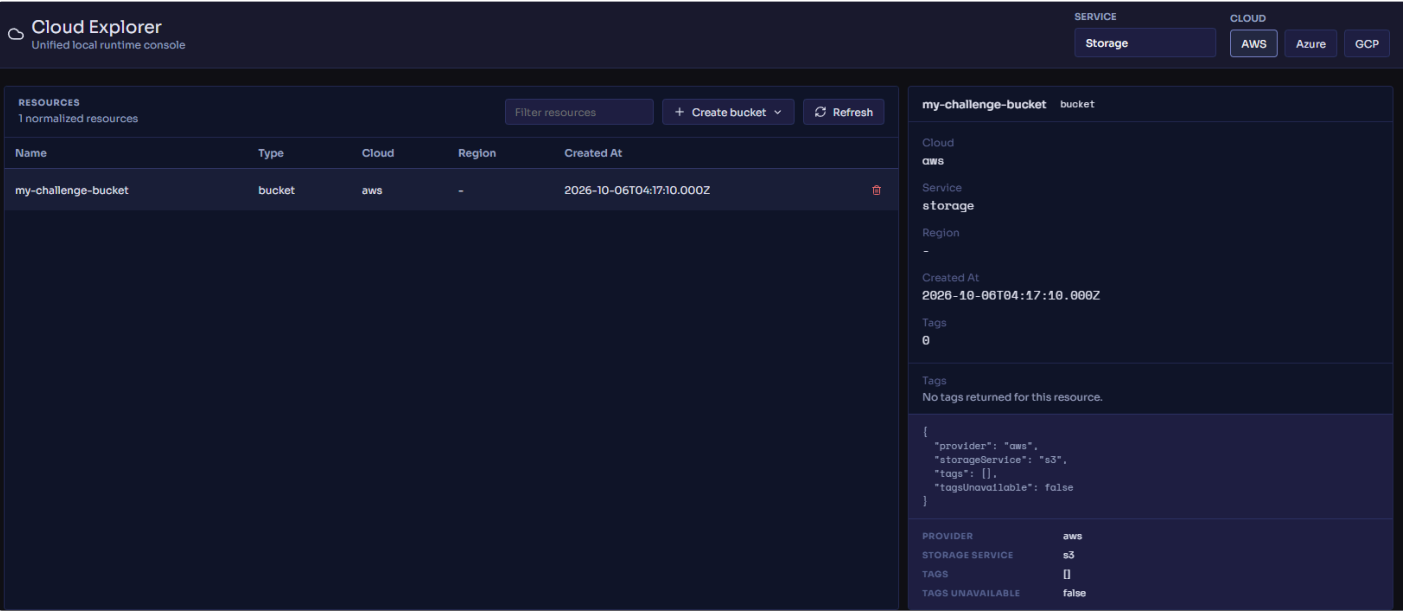


Figure 2. The floci Cloud Explorer GUI confirming the successful creation of the simulated AWS S3 bucket (my-challenge-bucket) and displaying its resource metadata.

Part 3: Completing the task

Key	Value
3.1 Access the object using a web browser	http://localhost:4566/my-challenge-bucket/hello.html
3.2 Listing the contents of the S3 bucket using the AWS CLI	aws --endpoint-url=http://localhost:4566 s3 ls s3://my-challenge-bucket

```
pichurn@LAPTOP-C67DMGAU:~$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the co
mmand 'aws login'.

AWS Access Key ID [None]: test
AWS Secret Access Key [None]: test
Default region name [None]: us-west-2
Default output format [None]: json
pichurn@LAPTOP-C67DMGAU:~$ aws --endpoint-url=http://localhost:4566 s3 ls s3://my-challenge-bucket
2026-10-06 12:17:11          39 hello.html
pichurn@LAPTOP-C67DMGAU:~$ |
```

Figure 3. Configuring the AWS CLI with mock credentials to interact with a local mock AWS environment (floci). The s3 ls command successfully queries localhost:4566 to list the hello.html object inside my-challenge-bucket.

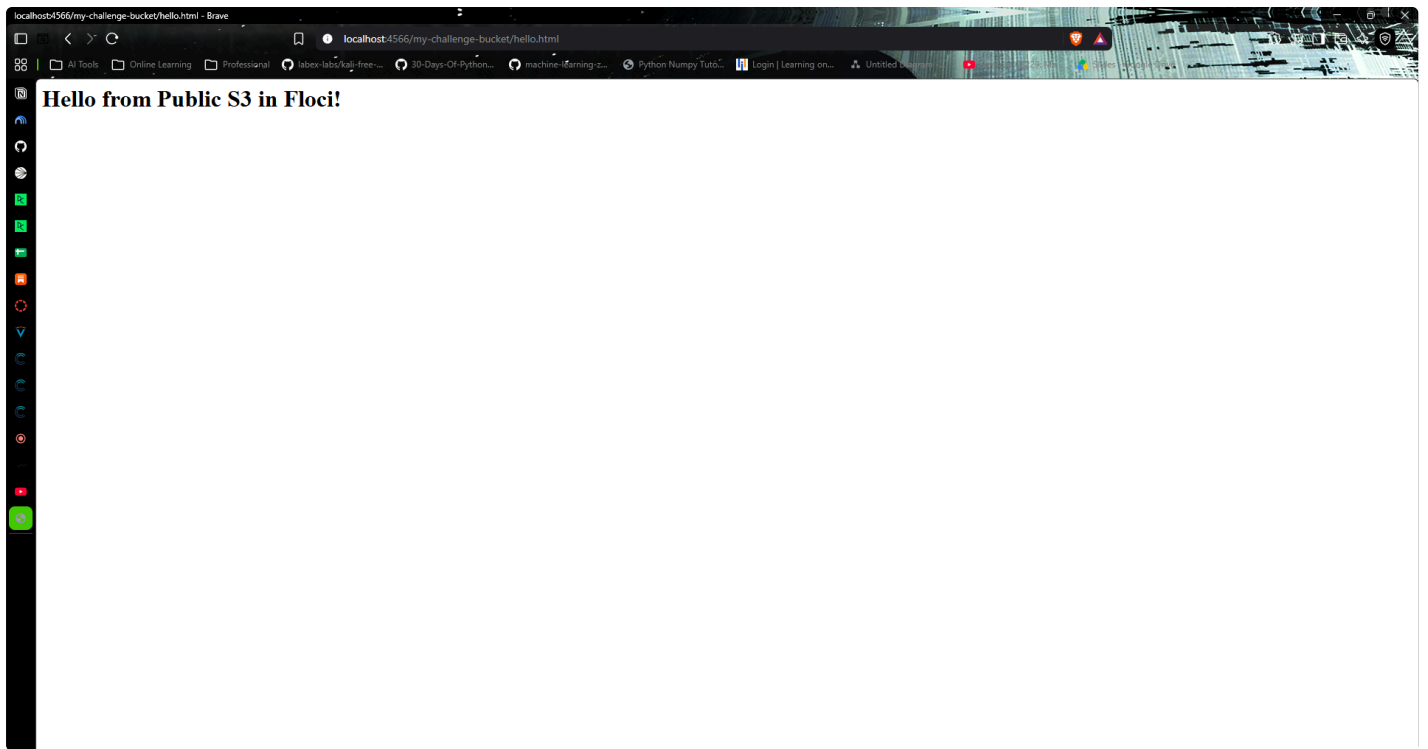


Figure 4. This verifies that the locally simulated S3 bucket is successfully configured to serve public static web content.